Compression d'images par Super Pixels

Mathys CHAMPENOIS, Cesar CHARBEY, Mickael CZOLACZ
Master IMAGINE

Contexte et Objectifs

- Pourquoi compresser ?
- L'idée : Utiliser la redondance spatiale et colorimétrique.
  Au lieu de stocker chaque pixel, on stocke des "régions homogènes"
- Problème : Comment réduire la taille d'une image sans dégrader sa cohérence visuelle ?

Plan

- Etats de l'art et choix technique
- Coeur de l'algorithme
- Connectivité
- Pipeline de compression : BitPacking
- Résultat et Analyses
- Démonstration et Conclusion

Etat de l'art et choix techniques

- Algorithme de référence : SLIC (Simple Linear Iterative Clustering) basé sur la méthode par Achanta et al.
- Avantage majeur : Complexité linéaire O(N), là où les méthodes précédentes étaient en $O(N^2)$.
- Article d'appui : Bilateral K-Means for Superpixel Computation, R. Gay et al. (IPOL 2022).
- Notre choix technique : L'article de Gay et al. démontre qu'une segmentation directement dans l'espace colorimétrique RGB (au lieu de CIELAB) permet de gagner en vitesse et en simplicité, tout en gardant une excellente fidélité pour la compression.

Coeur de l'algorithme SLIC :

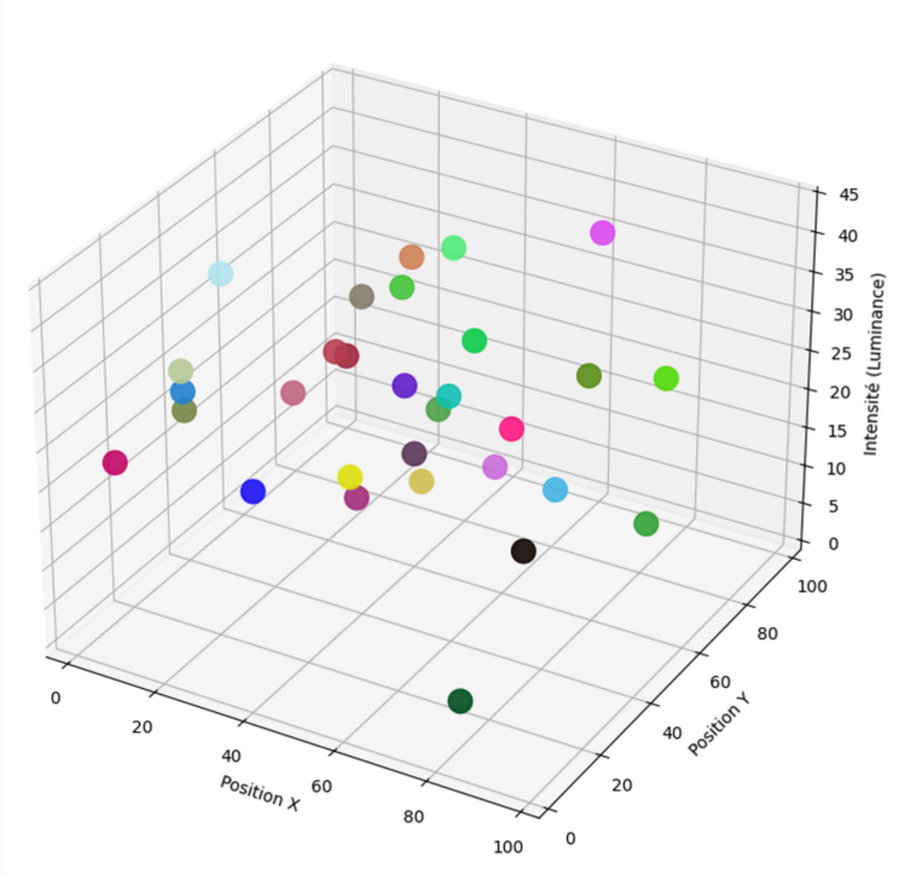
Le partitionnement (K-Means Bilatéral)

- Espace 5D : Chaque pixel est traité selon sa position et sa couleur : (x, y, r, g, b).

- Recherche locale : Au lieu de chercher sur toute l'image, le K-Means est restreint à une petite fenêtre, garantissant une exécution rapide.

- Paramètres clés gérés :

  K : Nombre cible de superpixels.
  m (Compacité) : Équilibre entre spatialité et colorimétrie.
  g : Perturbation initiale des centres via le gradient minimum.



Coeur de l'algorithme SLIC :

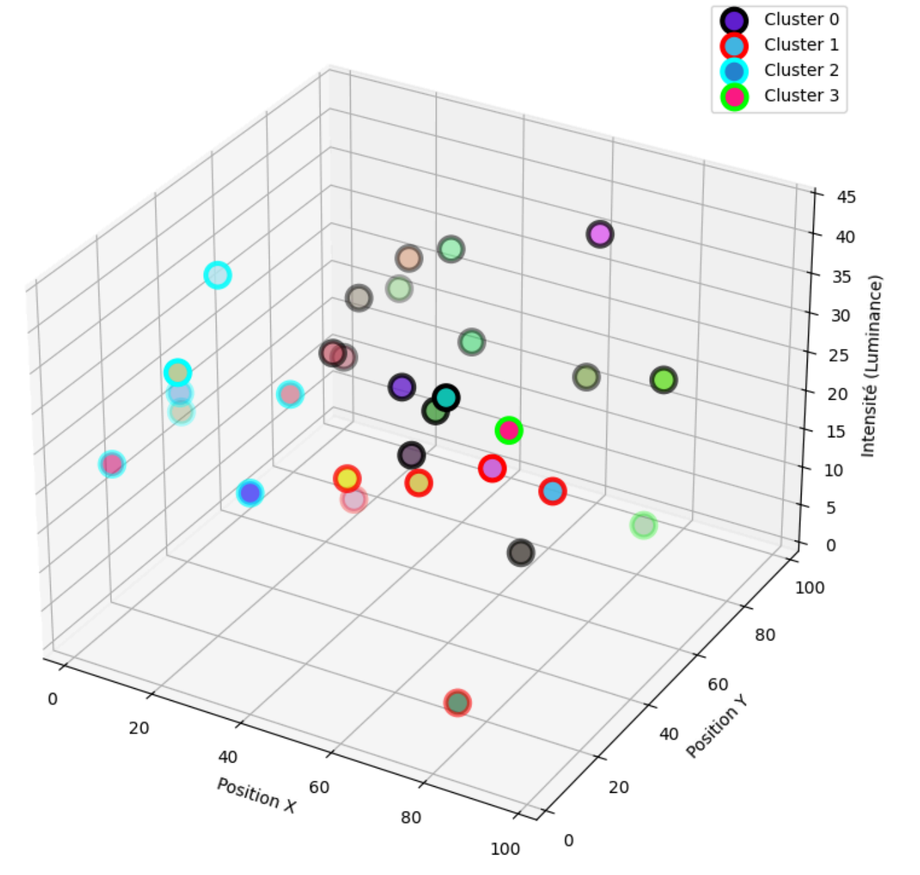
Le partitionnement (K-Means Bilatéral)

- Espace 5D : Chaque pixel est traité selon sa position et sa couleur : (x, y, r, g, b).

- Recherche locale : Au lieu de chercher sur toute l'image, le K-Means est restreint à une petite fenêtre, garantissant une exécution rapide.

- Paramètres clés gérés :

  K : Nombre cible de superpixels.
  m (Compacité) : Équilibre entre spatialité et colorimétrie.
  g : Perturbation initiale des centres via le gradient minimum.



La connectivité :

La problématique : Le K-Means local brise parfois les superpixels, créant des pixels isolés ou du bruit (orphelins)

La solution :

- <u>Étiquetage :</u> Détection des composantes connexes réelles via un parcours en largeur.
- <u>Graphe d'adjacence :</u> Cartographie des voisins de chaque région.
- <u>Isolation et Adoption :</u> Les petits fragments sont isolés, une distance de Manhattan est calculée, et ils sont adoptés par le voisin le plus proche en couleur.

→ Cette étape est cruciale pour garantir une carte parfaitement contiguë, indispensable à notre compression.

Compression par BitPacking

- Stockage des indices sur le nombre stricte de bits nécessaires
- RLE : exploitation de la redondance spatiale
- Fichier binaire optimisé contenant la grille et les labels compressés

Résultats : Impact de la Connectivité

(a) SLIC Brut (Fragments isolés)

(b) Résultat Propre (Après connectivité)

- Lors d'un test de contrainte (m = 5), la couleur domine et éclate les régions.
- Notre post-traitement parvient à tout réaffecter pour créer des zones stables, maximisant l'efficacité du RLE.

Performances de Compression

- Un facteur de compression d'environ 10x
- Taille originale : 3 275 536 octets → Compressée : 325 945 octets
- PSNR : 30.74 dB (K = 2048 et m = 2)

Image d'entrée

Image en sortie

Optimisation automatique : Auto-SLIC

Le défi :

- Trouver les bons paramètres (K , g , m) est chronophage pour l'utilisateur.

Notre solution :

- Développement d'un outil d'évaluation (autoSLIC.cpp).

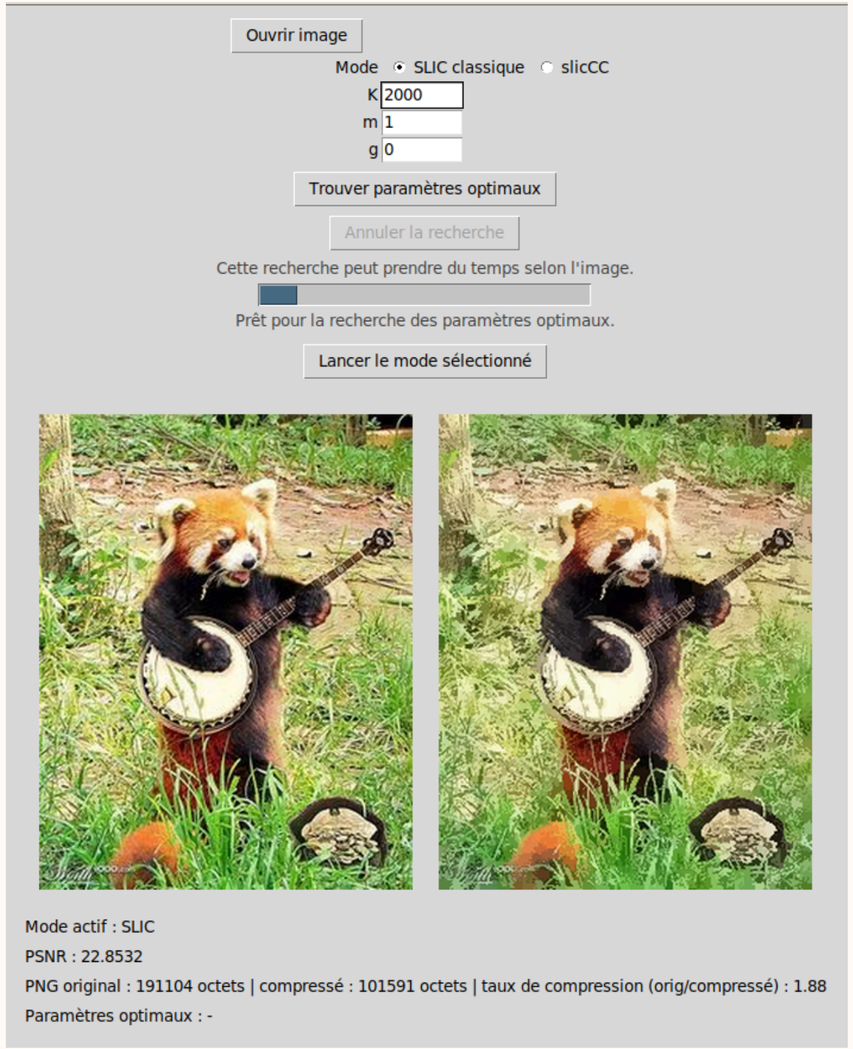
Fonctionnement :

- Utilisation d'un algorithme de Hill Climbing avec exécution parallèle.

L'algorithme cherche à maximiser le PSNR cible (ex : > 30 dB) tout en poussant le facteur de compression au maximum.

Interface Graphique :

- Interface graphique utilisant TKinter
- Lancement automatique du noyau C++ avec les paramètres dynamiques
- Calcul et affichage en temps réel du PSNR et de la taille du fichier compressé
- Option pour calculer les paramètres optimaux



Démo :

Conclusion

Implémentation fidèle et améliorée :

- Développement complet de SLIC en C++ avec un focus majeur sur la résolution des problèmes de connectivité.

Format sur mesure :

- Création du format .slic combinant astucieusement Bit-Packing et RLE.

Automatisation et Ergonomie :

- Un puissant outil de recherche de paramètres couplé à une interface utilisateur.